

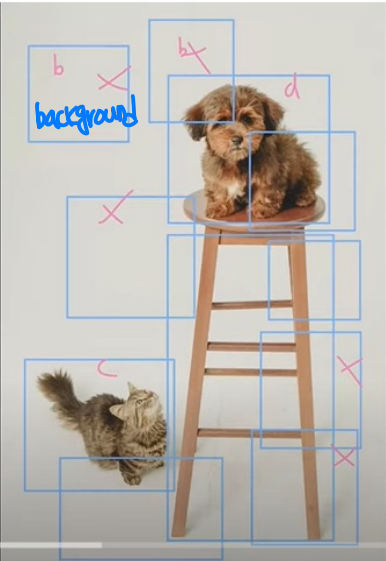


classical
 Region proposal (몇가지 후보를 제안함), $\rightarrow$ CNN을 돌림.
 2000개 box만

RCNN

Faster R CNN

$R \rightarrow CNN$
 Classification



~ 2000
 NN을 넣기 위해 resize
 224
 224
 Classification

Score 높은
 강아지, 고양이, 의자
 박스만

단점: 2000개의 box $\rightarrow$ CNN 돌려야 시간의
 오래걸림

Fast -RCNN

Faster R CNN

f_{net} R-CNN

~ 2000

The video player shows a presentation on Faster R-CNN. The main content is a video of a dog on a stool and a cat on the floor, with blue bounding boxes and red 'X' marks indicating object detection. The video player interface includes a red progress bar, play/pause buttons, a volume icon, a time display of 2:36 / 6:13, and a small inset video of a person in the bottom right corner.

VGG-19

3x3 conv, 64
3x3 conv, 64
3x3 conv, 128
3x3 conv, 128
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
fc 4096
fc 4096

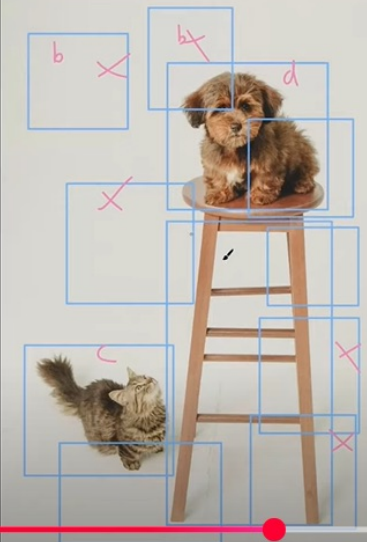
ResNet-152

7x7 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 128
3x3 conv, 128
152 layers
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512

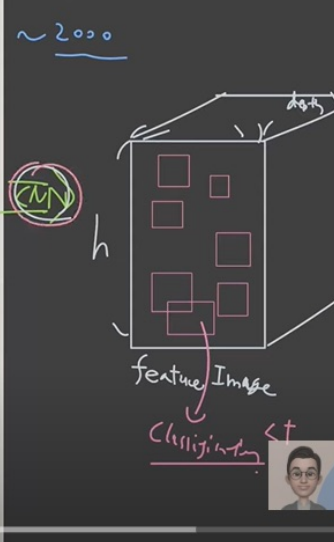
전체 이미지 CNN 적용 → feature 뽑아냄 → 공간정보 유지

Faster R CNN

$\text{Fast} + \text{R-CNN}$
 Classification



~ 2000







3:37 / 6:13



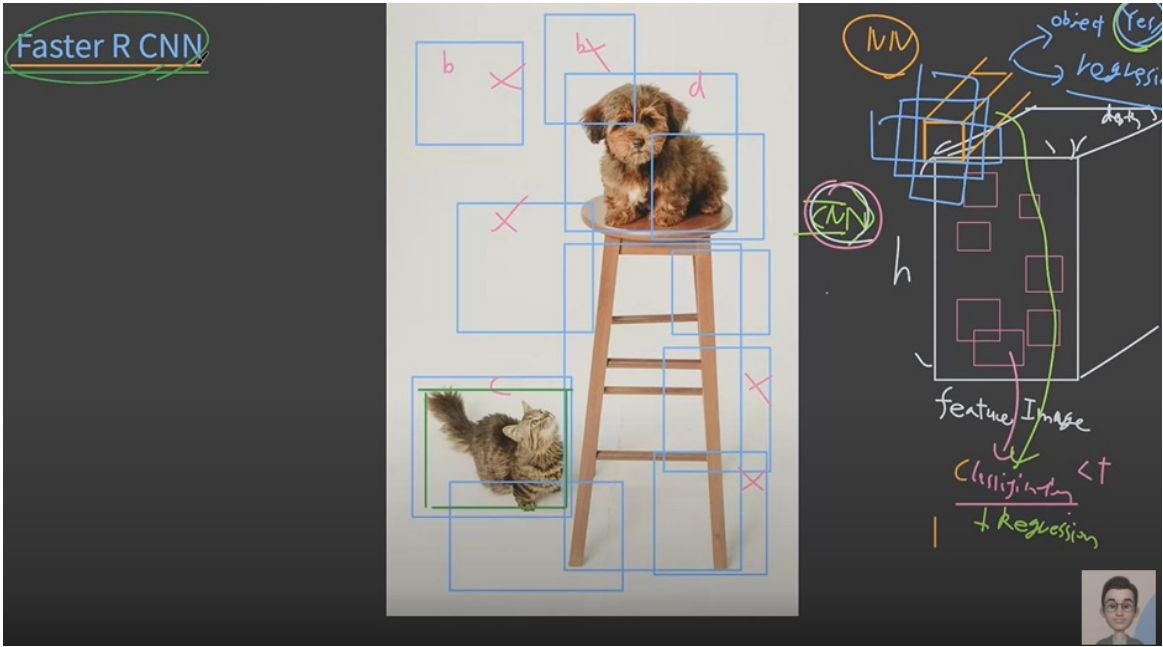









Faster RCNN



region proposal $\rightarrow$ CNN로 바꿈

관심있는 object가 있는가 없는가 $\rightarrow$ binary

관심있는 위치는 어디인가 $\rightarrow$ regression

anchor 박스 $\rightarrow$ 관심있는 물체(가)

어디에있고 어떻게
크기?

1

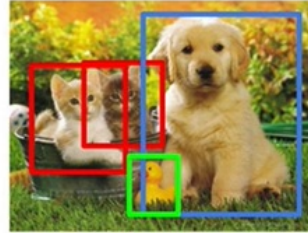
Object Detection

Classification



CAT

Object Detection

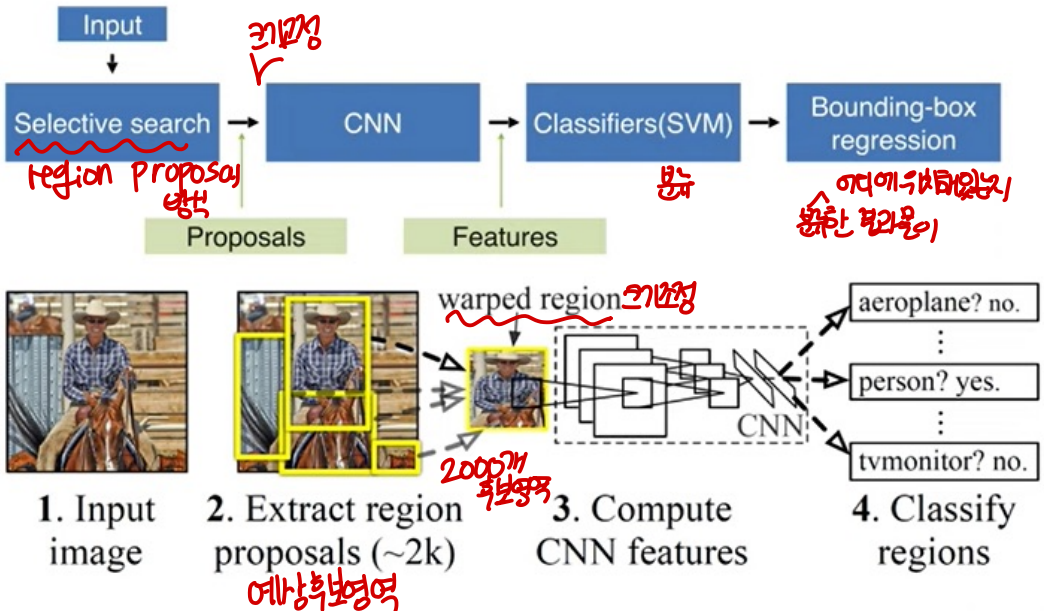


CAT, DOG, DUCK

f 이미지를 단순히 여러개
이러지않게
{ class가 어디에 위치하는지
판별

1

R-CNN



❖ 특징

▪ Selective Search

- 약 2000여개의 region proposal

후보 영역 생성

분류 (이전클래스와 다른)

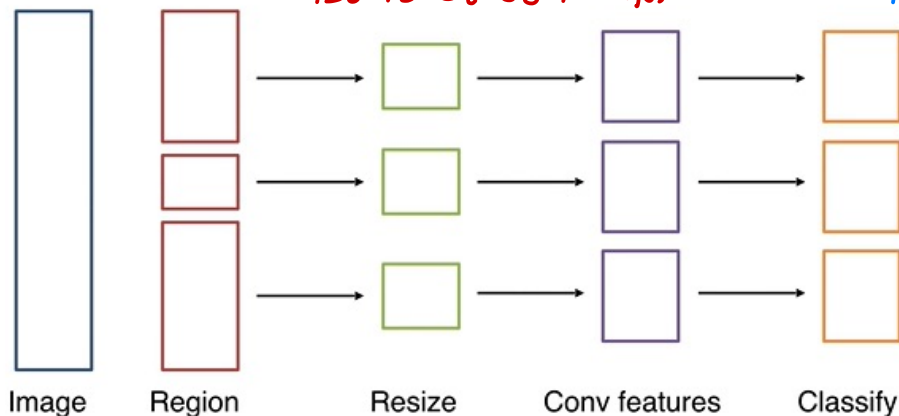
▪ Multi stage

- 세 단계로 학습이 진행 (Conv fine tune -> SVM classification -> BB regression)

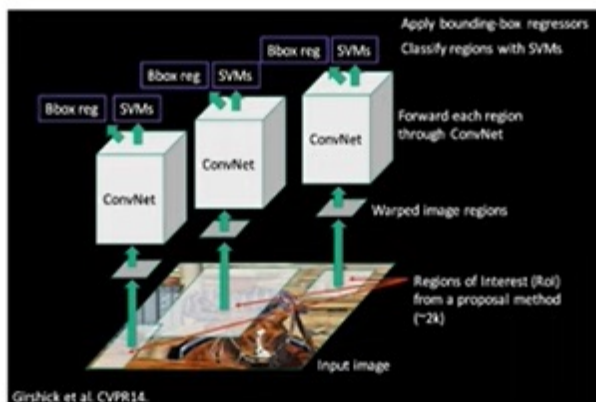
1단계
후보영역을 검색할 후보로 따져서

2단계

3단계

이제에 있어서 무비
파악

Fast R-CNN



Girshick et al. CVPR14.

❖ R-CNN 한계

- 합성곱 신경망의 입력을 위한 고정된 크기를 위해 ^{2차원} warping / crop을 사용해야 하며 그 과정에서 이미지의 정보 손실이 일어난다.
- 2000여개의 영역마다 CNN을 적용해야 하기에 학습 시간이 오래 걸린다.
- 학습이 여러 단계로 이루어지며 이로 인해 긴 학습 시간과 대용량 저장 공간이 요구된다.
- Object Detection의 속도 자체도 느리다.

Fast R-CNN

❖ R-CNN&SPPnet 의 한계

- 학습이 multi-stage로 진행됨
- 학습이 많은 시간과 저장 공간을 요구
- 실제 Object detection이 느리다

❖ Fast R-CNN을 통해서

- R-CNN&SPPnet과 비교하여 더 좋은 성능 획득
- single-stage로 진행되는 학습
- 전체 네트워크 업데이트 가능
- 저장 공간이 필요하지 않음
- 더 빠른 시간 (학습시간 9배/3배, 실제 시간 213배/10배 속도 개선)

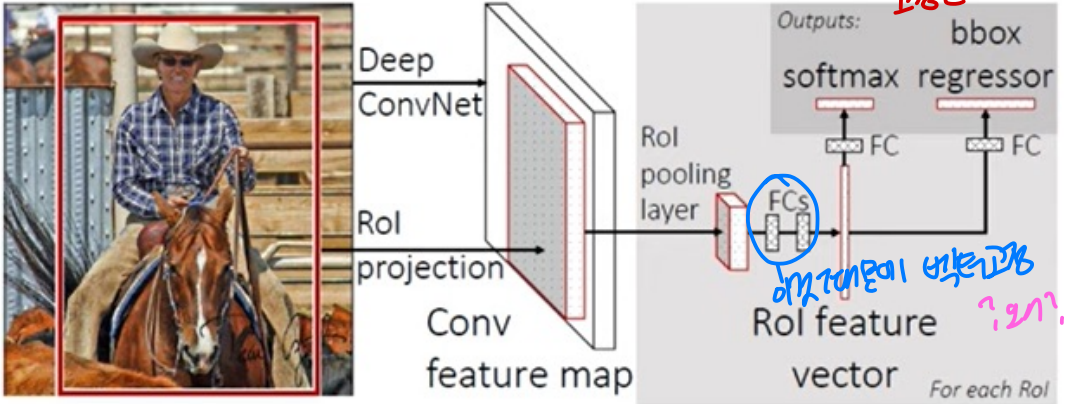
❖ Fast R-CNN 학습과정

- ① Input으로 이미지와 object proposal 사용
- ② 이미지를 통해 conv feature map 생성
- 각 object proposal로부터 RoI pooling layer를 통해 고정된 feature vector 생성
- FCN을 통해 object class를 판별 / bounding box를 조절

RCNN (selective search)

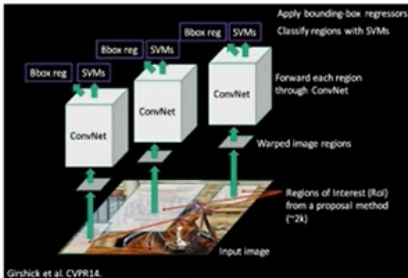
왜? Fully connected layer

이 이미지들만 파서 map을 object proposal을 통해 나온 feature vector를 고정된 vector로 만들



4

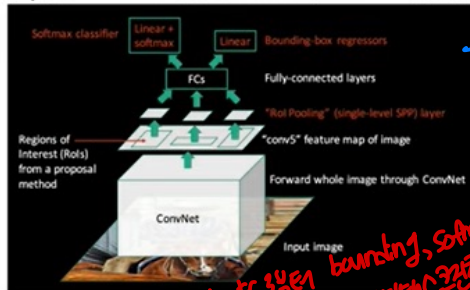
Fast R-CNN



RCNN 구조

Fast R-CNN 구조

SPPnet 구조



SPP를 사용

SPPnet처럼 이미지들만 추출한 후 학습 가능

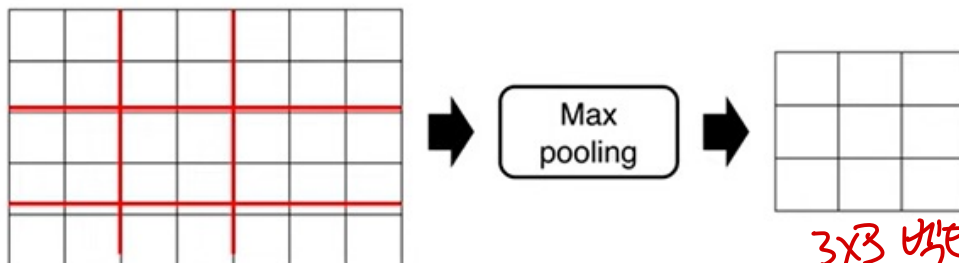
FC layer, bounding, softmax, RoI pooling layer 사용

Fast R-CNN

❖ RoI pooling layer

- RoI 영역에 해당하는 부분만 max pooling을 통해 feature map으로부터 고정된 길이의 저차원 벡터로 축소

- 각각의 RoI는 (r, c, h, w) 의 튜플 형태 (r, c) : top-left corner의 좌표
- $h * w$ RoI 사이즈를 $H * W$ 의 작은 윈도우 사이즈로 나눔 $\rightarrow h/H * w/W$
- SPPnet의 spp layer의 한 pyramid level만을 사용하는 형태와 동일



5x5 RoI 영역

3x3 벡터로 만든다.

4

Fast R-CNN

16

그 과정, 왜 그래?

❖ 학습을 위한 변경점

- 이미지넷을 사용한 선학습된 네트워크로 시작
- 마지막의 max pooling layer가 RoI pooling layer로 대체 (VGG16에서 $H=W=7$)
- 신경망의 마지막 fc layer와 softmax단이 두 개의 output layer로 대체 (원래는 이미지넷 1000개 분류)
- 신경망의 입력이 이미지와 RoI를 반영할 수 있도록 변경

과정 벡터로 만든다 (이해)

→ bounding box는 항상 2개 단위
바꿔줌

→ 이미지와 bounding box에 대한 RoI (selective search)를 찾아
인자로.

❖ Detection을 위한 Fine-tuning

- SPPnet & R-CNN은 CNN학습에서의 한계 (이미지에 대한 합성곱 역전파 학습이 불가)
→ 2000개 영역 전체에 대해 하나의 region proposal (찾은 구역)을
- Region-wise sampling $\rightarrow$ Hierarchical sampling
- SGD 미니배치가 이미지에 종속됨 (N개의 이미지를 각 R/N 개로)
- $N=2, R=128$ 로도 좋은 학습 가능 / 약 64배 빠른 학습 가능 (N이 작을수록 계산복잡도도 낮아진다)
- Single stage

2단계 (64x64x2)

→ 합성곱과 pooling 단계 (비트맵)와 영역에 대한 학습

- 최종 classifier와 regression 까지 단방향 단계

❖ Multi-task loss

- 두 개의 output layer **병렬태도**
- 분류 : 각 RoI별 Discrete probability distribution(전체 K+1 카테고리)
- 회귀 : bounding box regression $t^k = (t_x^k, t_y^k, t_w^k, t_n^k)$
- u, v : ground truth class & target

$$L(p, u, t^u, v) = \underbrace{L_{\text{cls}}(p, u)}_{\text{배경일 경우 0}} + \lambda [u \geq 1] \underbrace{L_{\text{loc}}(t^u, v)}_{\text{아웃나서 더 정확하게 맞추기 위해}}$$

$$L_{\text{cls}}(p, u) = -\log p_u$$

<분류>

$$L_{\text{loc}}(t^u, v) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i^u - v_i)$$

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

<회귀>

19

Fast R-CNN

❖ 미니배치 샘플링 **Hierarchical**

- N=2 로 미니배치 구성
- R=128로 각 이미지당 64개의 RoI 구축
- RoI의 25%는 전체 object proposal에서 IoU가 0.5 이상 **정확한 영역** $\rightarrow$ **실제 이미지 안에 클래스 포함**
- 그 이외는 IoU가 0.1에서 0.5 사이(배경 class) **negative**
- 학습 과정에서 이미지는 0.5의 확률로 수평으로 뒤집어진다.

❖ 스케일 불변성

- Brute force(single scale)
- Image pyramid(multi scale)

❖ Fast R-CNN detection

- 실제 환경에서 보통 2000개의 RoI, 224*224 스케일과 비슷하게 사용
- 각 RoI r마다 사후 class분포 값과 bb 예측값을 산출
- 각 class k마다 r에 대한 $\Pr(\text{class} = k | r)$ 값인 detection confidence 부여
- 각 class별 Non-maximum suppression 사용 → RoI 개수 줄임

❖ Truncated SVD for faster detection

- 빠른 감지를 위한 Truncated SVD 사용
- $U : u \times t, V : v \times t$
- 파라미터 수를 $uv \rightarrow t(u+v)$ 만큼 감소

$$W \approx U \Sigma_t V^T$$

❖ 실험 결과

- VOC07, 2010, 2012 에서 SOTA mAP
- R-CNN, SPPnet과 성능, 속도 비교
- VGG16에서 fine-tuning을 통한 mAP 향상
- 온라인에서 제공되는 학습된 ImageNet 모델 사용

	모델	특징
S	CaffeNet	AlexNet과 동일
M	VGG_CNN_M_1024	S와 깊이는 같지만 더 넓음
L	deep VGG16	깊은 신경망

❖ 결론

- R-CNN / SPPnet에 비해 더 빠르고 높은 성능
- One-stage 구조를 통한 효율적인 학습이 가능
- 충분히 빠른가의 문제가 여전히 남아 있다



Faster R-CNN

Faster R-CNN

❖ Fast R-CNN 한계

- Testing 단계에서 여전히 느리다
- Why? Region proposal 단계가 대부분의 시간을 차지하기 때문

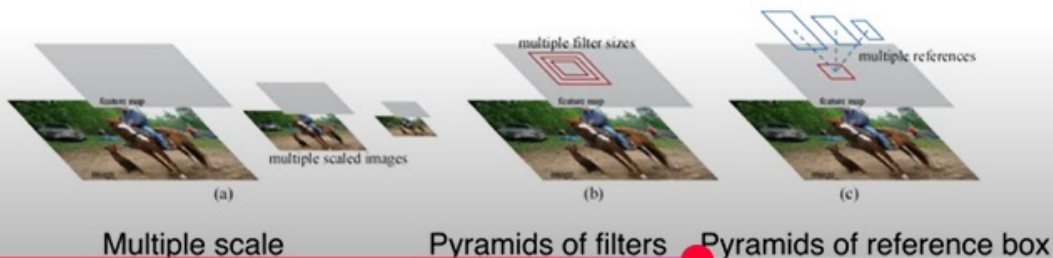


selective search → CPU에 많은 사용됨
 Region proposal에 신경망을 적용하여 시간을 단축하자 *GPU 사용함*
 = RPN(Region Proposal Network 도입)

Faster R-CNN

❖ Faster R-CNN의 의의

- Region proposal을 합성곱 신경망을 활용하여 CPU에서 GPU의 영역으로 전환
- Fast R-CNN과 같은 region-based 감지기의 feature map을 region-proposal generating에도 사용하자
전체 이미지 → 패치 맵 *→ 사용*
- RPN은 end to end로 학습이 가능하며 object여부와 bounding box를 regress하는 하나의 FCN

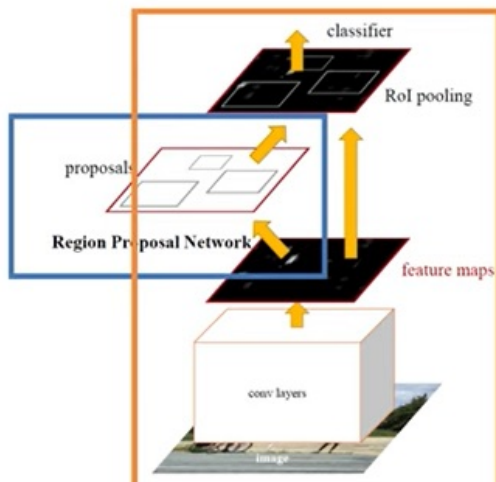


Faster R-CNN

❖ Faster R-CNN 구조

- 마치 어텐션처럼 RPN은 Fast R-CNN이 어디에 주목해야하는지 알려준다

→ Faster R-CNN

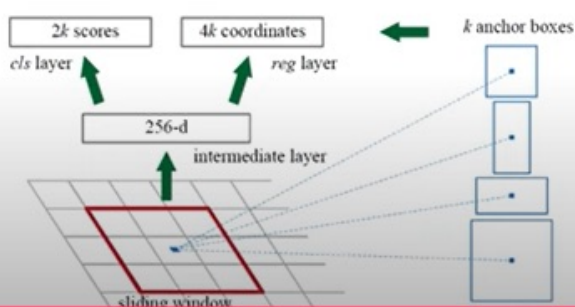


$$\boxed{\text{RPN}} + \boxed{\text{Fast RCNN}}$$

❖ Region Proposal Networks

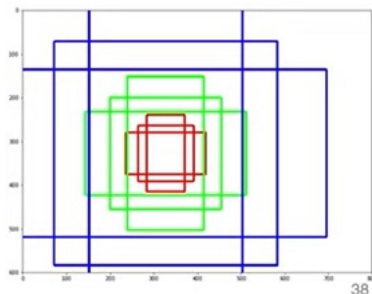
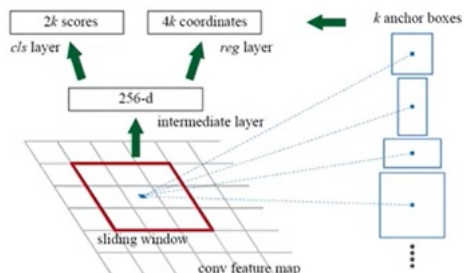
- 다양한 사이즈의 이미지를 입력 값으로 object score와 object proposal을 출력
- Fast r-cnn과 합성곱 신경망을 공유
- feature map의 마지막 conv 층을 작은 네트워크가 sliding하여 저차원으로 매핑
- Regression과 classification 수행

이렇게 바깥



❖ Region Proposal Networks- Anchors

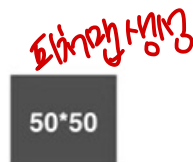
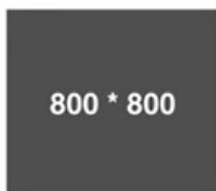
- 각 슬라이딩 윈도우에서 Bounding box의 후보로 사용되는 상자
- reg층은 4k를, cls층은 2k만큼의 출력을 갖게 된다.
- anchor는 슬라이딩 윈도우 중심에 있으며 미리 정해진 스케일과 비율에 따라 달라진다.
- 실험에서는 3개의 스케일과 3개의 비율은 사용하여 k=9개의 앵커를 사용하였다.
- W*H 크기만큼의 conv feature map에서 WHk만큼의 앵커가 있는 것이다.



Faster R-CNN

❖ Region Proposal Networks- Anchors

- Feature map에 해당하는 픽셀 단위 앵커 박스 생성



VGG16

```
In[11]: print(image.shape)
torch.Size([1, 3, 800, 800])
```

```
In[12]: print(out_map.size())
torch.Size([1, 512, 50, 50])
```

(8,8) anchor box
(3 scale * 3 ratio)

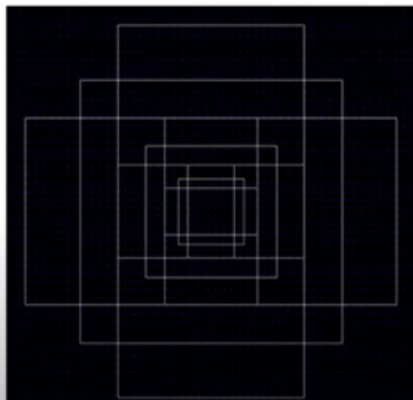
```
In[8]: print(anchor_base)
[[ -37.254833  -82.50967    53.254833   98.50967 ]
 [  -82.50967  -173.01933   98.50967   189.01933 ]
 [ -173.01933  -354.03867   189.01933   370.03867 ]
 [   -56.      -56.         72.         72.        ]
 [  -120.      -120.        136.        136.        ]
 [  -248.      -248.        264.        264.        ]
 [  -82.50967  -37.254833   98.50967   53.254833 ]
 [ -173.01933  -82.50967   189.01933   98.50967 ]
 [ -354.03867  -173.01933   370.03867   189.01933 ]]
```

39

Faster R-CNN

❖ Region Proposal Networks- Anchors

- HWk에 해당하는 앵커박스 생성 및 가장자리를 벗어나지 않는 유효한 앵커 박스 사용



```
In[16]: print(anchors.shape)
....: #50x50개의 중심점에 대해 9개의 앵커박스 생성
....:
(22500, 4)
```

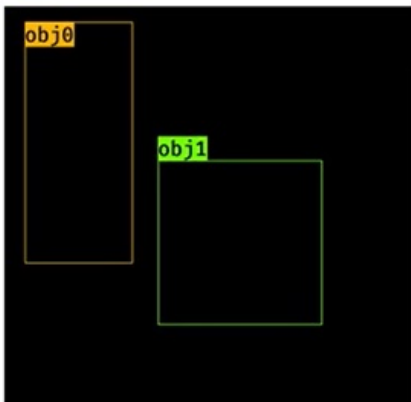


```
In[18]: print(index_inside.shape)
....: #가장자리를 벗어나지 않는 유효한 앵커 박스
....:
....:
(8940,)
```


❖ Region Proposal Networks- Anchors

- Bounding Box와 가장 높은 IoU를 갖는 앵커박스 확인
- 각 앵커박스에 대해서 Bbox와의 IoU값이 큰 앵커박스 확인
- Labeling 실시

맞음! 아니면 0
2배감



```
In[27]:
...: max_iou = ious[np.arange(len(index_inside)), argmax_iou]
...: print(max_iou)
...: #각 앵커박스에 대해서 Gt와의 가장 큰 iou값
...:
[10.980879  8.354239  5.874238 ... 0.40959978 0.40959978
 0.40959978]
```

```
In[28]:
...: gt_argmax_iou = np.where(iou == gt_max_iou)[0]
...: print(gt_argmax_iou)
...: #Gt와 가장 높은 IoU값을 갖는 앵커박스의 위치
...:
[  0  38  41 118 198 350 4831 4839 4847 5097 5104 5111
 5357 5589 5596 5603 5835 5842 5849]
```

```
label[max_iou < neg_iou_threshold] = 0
label[gt_argmax_iou] = 1
label[max_iou >= pos_iou_threshold] = 1
```

❖ Translation-Invariant(이동불변성) Anchors

- RPN에서 윈도우 크기를 슬라이딩 하는 방식은 이동불변성을 보장
- CNN에서 윈도우 슬라이딩을 사용하여 convolution을 하였을때 얻는 효과와 동일
- 모델 크기를 줄여준다 ($(4+1)*800$ vs $(4+2)*9$) -> 연산량 절감

Translation Invariance



같은 특징을 뽑아줌
이동이 없다든

❖ Multi-scale Anchors as Regression References

- Pyramid of anchors
- 다양한 scale과 ratio를 활용한 anchor를 통해 효율적으로 계산할 수 있다.

멀티 스케일
SPN과 이분기 다양하게 진행 (직접 필터 사용 중요하게 생각)

앵커박스의 파라미터 조정하면? 필터나 사이즈 안!
조정가능



❖ RPN Loss function

- RPN 학습 : object인지 아닌지
- Positive : 가장 높은 IOU / IOU 0.7 이상 Negative : IOU 0.3 이하
- Fast R-CNN의 loss를 사용
- i: index of anchor, : object일 확률값, : object일 실제값
- R : smooth L1
- : 예측 값, : anchor box, : ground truth

$$L(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*) + \lambda \frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)$$

$$L_{reg}(t_i, t_i^*) = R(t_i - t_i^*)$$

$$\begin{aligned} t_x &= (x - x_a)/w_a, & t_y &= (y - y_a)/h_a \\ t_w &= \log(w/w_a), & t_h &= \log(h/h_a) \\ t_x^* &= (x^* - x_a)/w_a, & t_y^* &= (y^* - y_a)/h_a \\ t_w^* &= \log(w^*/w_a), & t_h^* &= \log(h^*/h_a) \end{aligned}$$

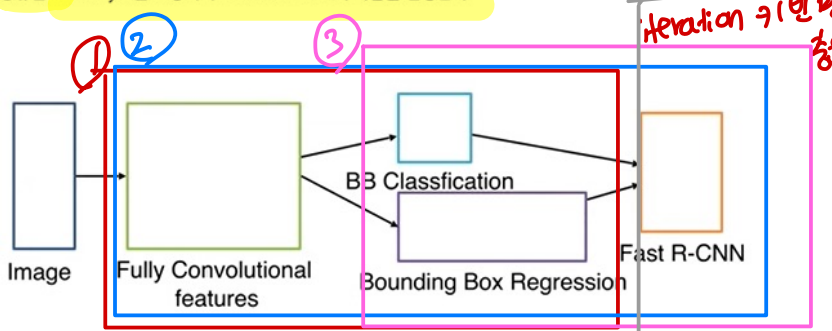
실제 object 인지 여부 판별

사용한 ROI의 개수

사용한 미니배치의 개수

❖ Faster R-CNN 학습 과정 : RPN과 Fast R-CNN의 결합

1. RPN은 ImageNet을 사용하여 학습된 모델로부터 초기화되며 region proposal task를 위해 end to end로 학습된다.
2. 위 단계에서 학습된 RPN을 사용하여 Fast R-CNN 모델의 학습을 진행한다. (초기화는 ImageNet 학습 모델로)
3. 초기화를 위의 네트워크를 사용하여 RPN을 학습하는데 공통된 Conv layer는 고정하고 RPN에만 연결된 층만 학습한다. → 전체적인 패치 공유
4. 공유된 conv layer를 고정시키고 Fast R-CNN의 학습을 진행한다.



❖ 실험

- PASCAL VOC 2007 사용하여 학습(5000/5000/20) + PASCAL VOC 2012 사용
- ZF 모델과 VGG-16 모델 사용

Table 2: Detection results on **PASCAL VOC 2007 test set** (trained on VOC 2007 trainval). The detectors are Fast R-CNN with ZF, but using various proposal methods for training and testing.

train-time region proposals		test-time region proposals		mAP (%)
method	# boxes	method	# proposals	
SS	2000	SS	2000	58.7
EB	2000	EB	2000	58.6
RPN+ZF, shared	2000	RPN+ZF, shared	300	59.9
<i>ablation experiments follow below</i>				
RPN+ZF, unshared	2000	RPN+ZF, unshared	300	58.7
SS	2000	RPN+ZF	100	55.1
SS	2000	RPN+ZF	300	56.8
SS	2000	RPN+ZF	1000	56.3
SS	2000	RPN+ZF (no NMS)	6000	55.2
SS	2000	RPN+ZF (no cls)	100	44.6
SS	2000	RPN+ZF (no cls)	300	51.4
SS	2000	RPN+ZF (no cls)	1000	55.8
SS	2000	RPN+ZF (no reg)	300	52.1
SS	2000	RPN+ZF (no reg)	1000	51.3
SS	2000	RPN+VGG	300	59.2

48

Faster R-CNN

❖ 실험-RPN 효과

- Conv 공유가 성능 향상을 가져온다(2번째 단계 후 멈춤)
- RPN은 적은 수의 proposal에도 강건하며 NMS가 성능 개선에 도움이 된다
- Cls, Reg loss가 모두 성능 향상에 영향을 준다
- Multi task 가 도움이 되며 깊은 모델이 좋은 성능에 영향을 끼친다

Table 2: Detection results on **PASCAL VOC 2007 test set** (trained on VOC 2007 trainval). The detectors are Fast R-CNN with ZF, but using various proposal methods for training and testing.

train-time region proposals		test-time region proposals		mAP (%)
method	# boxes	method	# proposals	
SS	2000	SS	2000	58.7
EB	2000	EB	2000	58.6
RPN+ZF, shared	2000	RPN+ZF, shared	300	59.9
<i>ablation experiments follow below</i>				
RPN+ZF, unshared	2000	RPN+ZF, unshared	300	58.7
SS	2000	RPN+ZF	100	55.1
SS	2000	RPN+ZF	300	56.8
SS	2000	RPN+ZF	1000	56.3
SS	2000	RPN+ZF (no NMS)	6000	55.2
SS	2000	RPN+ZF (no cls)	100	44.6
SS	2000	RPN+ZF (no cls)	300	51.4
SS	2000	RPN+ZF (no cls)	1000	55.8
SS	2000	RPN+ZF (no reg)	300	52.1
SS	2000	RPN+ZF (no reg)	1000	51.3
SS	2000	RPN+VGG	300	59.2

적은 수의
proposal에도
강건
성능
향상
같은 사용해서 성능

Faster R-CNN

❖ 실험-VGG-16 모델 성능

- PASCAL VOC2007, 2012를 활용해서 좋은 성능을 보인다
- SS보다 훨씬 빠른 속도/ 높은 성능을 보인다

Table 3: Detection results on **PASCAL VOC 2007 test set**. The detector is Fast R-CNN and VGG-16. Training data: "07": VOC 2007 trainval, "07+12": union set of VOC 2007 trainval and VOC 2012 trainval. For RPN, the train-time proposals for Fast R-CNN are 2000. †: this number was reported in [2]; using the repository provided by this paper, this result is higher (68.1).

method	# proposals	data	mAP (%)
SS	2000	07	66.9 [†]
SS	2000	07+12	70.0
RPN+VGG, unshared	300	07	68.5
RPN+VGG, shared	300	07	69.9
RPN+VGG, shared	300	07+12	73.2
RPN+VGG, shared	300	COCO+07+12	78.8

Table 5: **Timing** (ms) on a K40 GPU, except SS proposal is evaluated in a CPU. "Region-wise" includes NMS, pooling, fully-connected, and softmax layers. See our released code for the profiling of running time.

model	system	conv	proposal	region-wise	total	rate
VGG	SS + Fast R-CNN	146	1510	174	1830	0.5 fps
VGG	RPN + Fast R-CNN	141	10	47	198	5 fps
ZF	RPN + Fast R-CNN	31	3	25	59	17 fps

50

Faster R-CNN

❖ 실험-하이퍼파라미터 강건성

- 다양한 크기의 앵커를 사용하는 것이 좋다
- λ 에 대해 강건한 결과를 보인다

↑
높아

Table 8: Detection results of Faster R-CNN on PASCAL VOC 2007 test set using **different settings of anchors**. The network is VGG-16. The training data is VOC 2007 trainval. The default setting of using 3 scales and 3 aspect ratios (69.9%) is the same as that in Table 3.

settings	anchor scales	aspect ratios	mAP (%)
1 scale, 1 ratio	128 ²	1:1	65.8
	256 ²	1:1	66.7
1 scale, 3 ratios	128 ²	{2:1, 1:1, 1:2}	68.8
	256 ²	{2:1, 1:1, 1:2}	67.9
3 scales, 1 ratio	{128 ² , 256 ² , 512 ² }	1:1	69.8
3 scales, 3 ratios	{128 ² , 256 ² , 512 ² }	{2:1, 1:1, 1:2}	69.9

Table 9: Detection results of Faster R-CNN on PASCAL VOC 2007 test set using **different values of λ** in Equation (1). The network is VGG-16. The training data is VOC 2007 trainval. The default setting of using $\lambda = 10$ (69.9%) is the same as that in Table 3.

λ	0.1	1	10	100
mAP (%)	67.2	68.9	69.9	69.1

Faster R-CNN

❖ 실험-Recall-to-IoU

- SS, EB에 비해서 강건한 recall 값
- Proposal 개수가 적어지더라도 좋은 recall 값을 보여줌

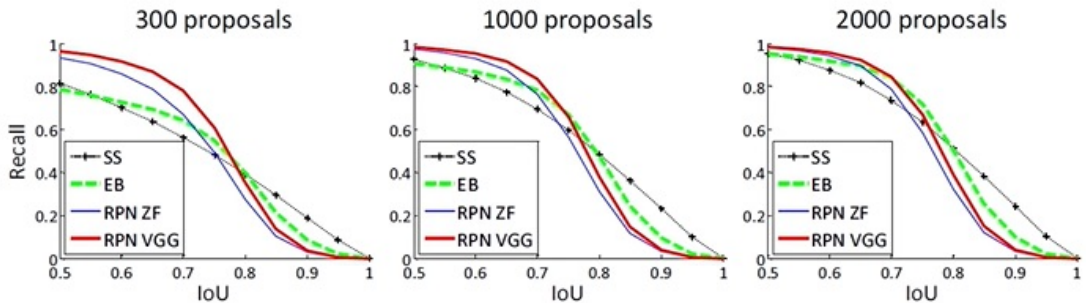


Figure 4: Recall vs. IoU overlap ratio on the PASCAL VOC 2007 test set.

52

Faster R-CNN

이름으로 미루어 proposal과 detection이 같이 진행?

❖ 실험-One stage Detection vs Two-stage Proposal+Detection

- Overfeat : one stage, class-specific
- Faster R-CNN : two-stage
- Overfeat 방법론에 비해서 더 빠르고 좋은 성능을 보임

Table 10: One-Stage Detection vs. Two-Stage Proposal + Detection. Detection results are on the PASCAL VOC 2007 test set using the ZF model and Fast R-CNN. RPN uses unshared features.

	proposals		detector	mAP (%)
Two-Stage	RPN + ZF, unshared	300	Fast R-CNN + ZF, 1 scale	58.7
One-Stage	dense, 3 scales, 3 aspect ratios	20000	Fast R-CNN + ZF, 1 scale	53.8
One-Stage	dense, 3 scales, 3 aspect ratios	20000	Fast R-CNN + ZF, 5 scales	53.9

❖ 실험-MS COCO dataset

- MS COCO데이터에서도 RPN을 활용했을시 더 좋은 성능을 보인다.

Table 11: Object detection results (%) on the **MS COCO** dataset. The model is VGG-16.

method	proposals	training data	COCO val		COCO test-dev	
			mAP@.5	mAP@[.5, .95]	mAP@.5	mAP@[.5, .95]
Fast R-CNN [2]	SS, 2000	COCO train	-	-	35.9	19.7
Fast R-CNN [impl. in this paper]	SS, 2000	COCO train	38.6	18.9	39.3	19.3
Faster R-CNN	RPN, 300	COCO train	41.5	21.2	42.1	21.5
Faster R-CNN	RPN, 300	COCO trainval	-	-	42.7	21.9



5

Faster R-CNN

❖ 실험-from MS COCO to PASCAL VOC

- COCO 데이터를 추가로 학습할 경우 성능 향상을 이루어냈다.
- 속도 또한 크게 느려지지 않았다.

Table 12: Detection mAP (%) of Faster R-CNN on PASCAL VOC 2007 test set and 2012 test set using different training data. The model is VGG-16. “COCO” denotes that the COCO trainval set is used for training. See also Table 6 and Table 7.

training data	2007 test	2012 test
VOC07	69.9	67.0
VOC07+12	73.2	-
VOC07++12	-	70.4
COCO (no VOC)	76.1	73.0
COCO+VOC07+12	78.8	-
COCO+VOC07++12	-	75.9

Table 6: Results on PASCAL VOC 2007 test set with Fast R-CNN detectors and VGG-16. For RPN, the train-time proposals for Fast R-CNN are 2000. RPN* denotes the unsharing feature version.

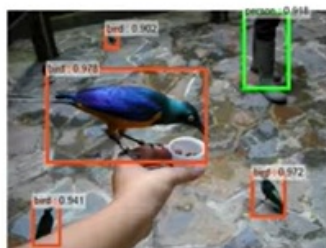
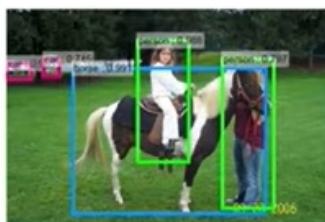
method	# bus	data	mAP	one	like	bird	bus	bottle	car	cat	chair	cup	table	dog	horse	inhibe	person	sheep	sofa	train	tv		
SS	2000	07	66.9	74.5	78.3	69.2	53.2	36.6	77.3	78.2	82.0	40.7	72.7	67.9	79.6	79.2	73.0	69.0	30.1	65.4	70.2	75.8	65.6
SS	2000	07+12	70.0	77.0	78.1	69.3	59.4	38.3	81.6	78.6	86.7	42.8	78.8	68.9	84.7	82.0	76.6	69.9	31.8	70.1	74.8	80.4	70.4
RPN ⁺	300	07	68.5	74.1	77.2	67.7	53.9	31.0	75.1	78.2	78.9	50.7	78.0	61.1	79.1	81.9	72.2	75.9	37.4	71.4	62.5	77.4	66.4
RPN	300	07	69.9	74.0	80.6	67.1	57.9	49.9	78.2	80.4	82.2	53.3	67.2	80.3	79.8	75.0	76.3	39.1	68.2	67.3	81.1	67.6	66.9
RPN	300	07+12	73.2	76.5	79.0	72.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.0	80.3	72.6
RPN	300	COCO+07+12	75.8	78.3	82.0	72.7	68.9	65.7	88.1	88.4	88.9	61.3	86.3	70.9	85.9	87.6	80.1	82.3	51.6	78.4	75.8	86.6	76.8

Table 7: Results on PASCAL VOC 2012 test set with Fast R-CNN detectors and VGG-16. For RPN, the train-time proposals for Fast R-CNN are 2000.

method	# bos	data	mAP	antelope	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	ship	soda	train	tv		
SS	2000	12	65.7	80.3	74.4	66.9	46.9	37.7	73.9	68.6	87.7	41.7	71.1	51.1	86.0	77.8	79.8	62.8	35.1	65.3	76.4	61.7	
SS	2000	07+12	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	72.0	32.1	65.3	65.3	78.4	64.2	
RPN	300	12	67.0	82.3	78.4	71.0	48.4	45.2	72.1	72.3	87.3	42.2	73.7	50.0	86.8	78.7	78.4	77.4	34.5	70.1	57.1	77.1	58.9
RPN	300	07+12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	73.9	88.5	45.1	74.7	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
RPN	300	COCO-07+12	75.9	87.4	83.6	76.8	62.9	59.6	81.9	82.0	91.3	54.9	82.6	59.0	89.0	85.5	84.7	84.1	32.2	78.8	65.5	85.4	70.6

❖ 결론

- RPN을 통해 cost-free한 region proposal 방법론을 제안하였다.
- 빠른 속도와 높은 정확도의 object detection을 이루어냈다.



6

Overall

<R-CNN>

Region proposal (Selective Search)	
Feature extraction (DNN)	
Classification (SVM)	Bounding box (Regression)

<Fast R-CNN>

Region proposal (Selective Search)
Feature extraction Classification + BB Regression (DNN)

<Faster R-CNN>

Region proposal Feature extraction Classification + BB Regression (DNN)
